Unit-2

Search Strategies and Problem Solving

Agent formulate a problem?

In artificial intelligence (AI) and machine learning, an agent is an entity that perceives its environment, processes information and acts upon that environment to achieve specific goals. The process by which an agent formulates a problem is critical, as it lays the foundation for the agent's decision-making and problem-solving capabilities.

Understanding Problem Formulation

Problem formulation is the process by which an agent defines the task it needs to solve. This involves specifying the initial state, goal state, actions, constraints, and the criteria for evaluating solutions. Effective problem formulation is crucial for the success of the agent in finding optimal or satisfactory solutions.

Steps in Problem Formulation

Define the Initial State: The initial state is the starting point of the agent. It includes all the relevant information about the environment that the agent can perceive and use to begin the problem-solving process.

Example: In a navigation problem, the initial state could be the agent's starting location on a map.

Specify the Goal State: The goal state defines the desired outcome that the agent aims to achieve. It represents the condition or set of conditions that signify the completion of the task.

Example: For the navigation problem, the goal state is the destination location.

Determine the Actions: Actions are the set of operations or moves that the agent can perform to transition from one state to another. Each action should be well-defined and feasible within the given environment.

Example: In a robot navigation scenario, actions could include moving forward, turning left, or turning right.

Establish the Transition Model: The transition model describes how the environment changes in response to the agent's actions. It defines the rules that govern state transitions.

Example: In a game, the transition model would include the rules that specify how the game state changes based on the player's moves.

Set Constraints and Conditions: Constraints are the limitations or restrictions within which the agent must operate. These can include physical limitations, resource constraints, and safety requirements.

Example: For a delivery drone, constraints might include battery life, weight capacity, and no-fly zones.

Define the Cost Function (if applicable): The cost function evaluates the cost associated with different actions or paths. It helps the agent to optimize its strategy by minimizing or maximizing this cost.

Example: In route planning, the cost function could represent the distance traveled, time taken, or energy consumed.

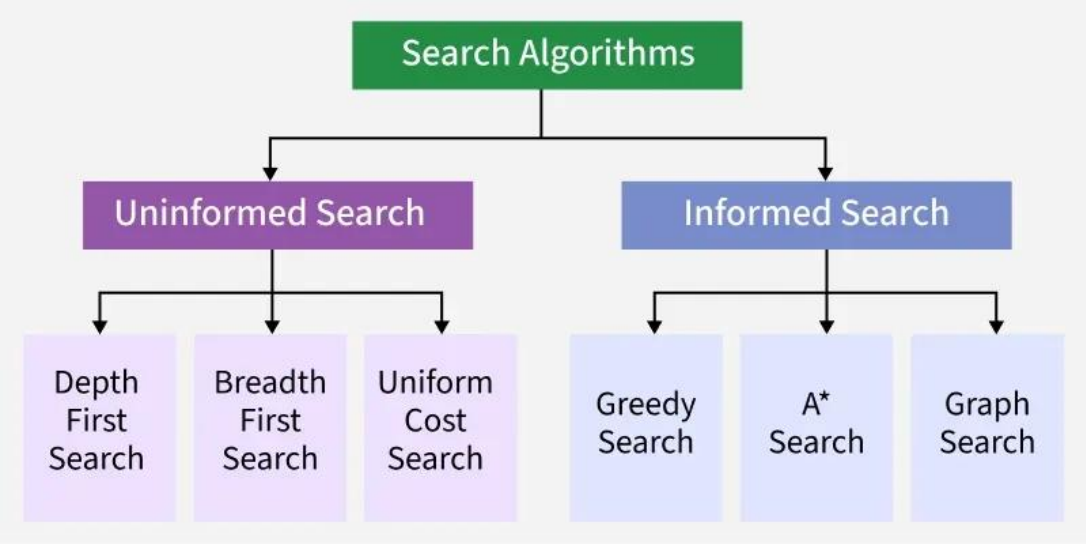
Criteria for Success: The criteria for success determine how the agent evaluates its progress and final solution. This includes metrics for measuring the effectiveness and efficiency of the solution.

Example: For a puzzle-solving agent, success criteria could be the completion of the puzzle within the shortest time or the fewest moves.

Search Algorithms in AI

Search algorithms in AI help find solutions by exploring possible paths or options in a problem space. AI uses them in tasks like path finding, decision making and game playing. These algorithms work by searching through

Types of search algorithms



There are mainly 2 types of search algorithms i.e Uninformed Search Algorithms and Informed Search Algorithms.

Uninformed Search Algorithms

Uninformed search also called blind search explores the search space without any domain specific knowledge or heuristics. It treats all nodes equally and chooses which path to explore next based solely on general rules like node depth or path cost.

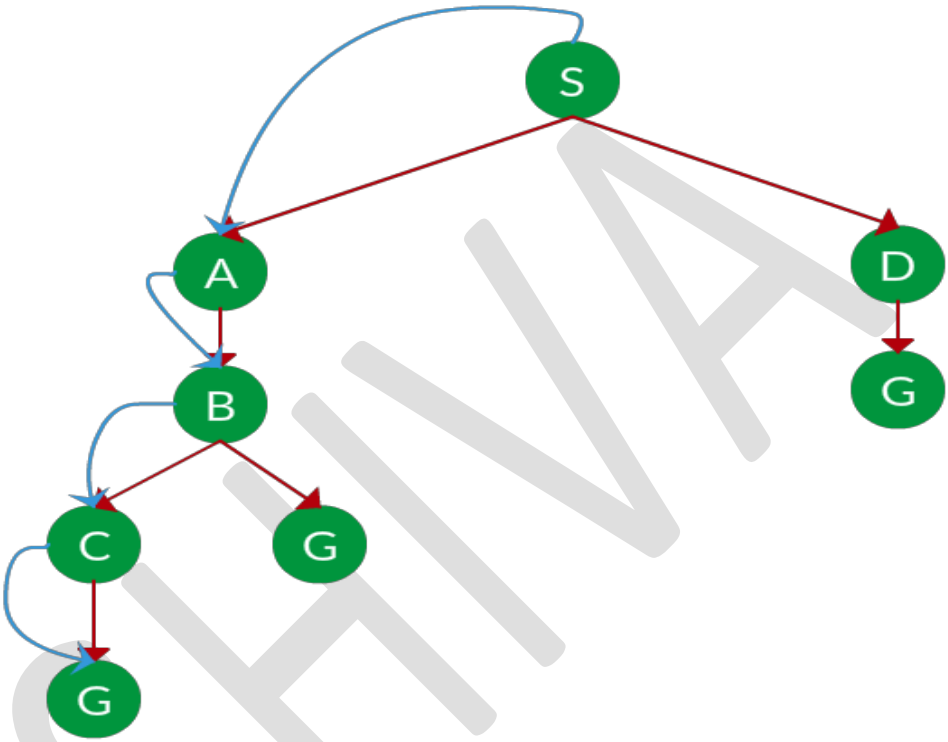
1. Depth First Search (DFS):

Depth First Search explores paths by going as deep as possible along one direction before backtracking. It uses a stack or recursion to keep track of the path.

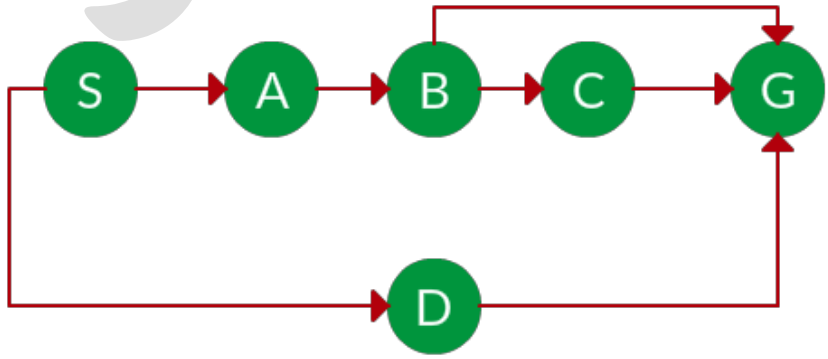
DFS is memory efficient compared to BFS since it doesn't need to store all siblings at each level.

However it is not guaranteed to find the shortest path and may get stuck in an infinite loop if the search tree is deep or contains cycles unless depth limits or visited checks are applied.

For Example: Which solution would DFS find to move from node S to node G if run on the graph below.



SHIVA

- 

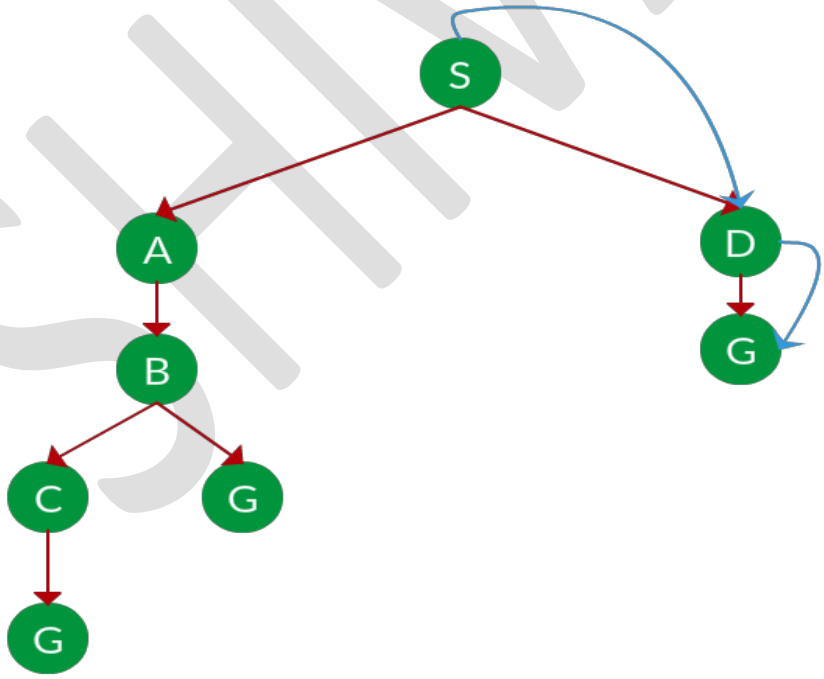
As DFS traverses the tree deepest node first it would always pick the

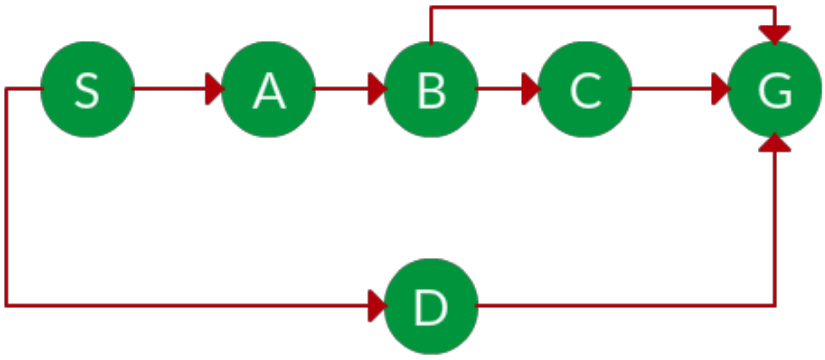
deeper branch until it reaches the solution or it runs out of nodes and goes to the next branch.

- Path: S->A->B->C->G

2. Breadth First Search (BFS):

- Breadth First Search is a fundamental search algorithm that explores all possible paths level by level. It begins from the root node and explores all neighboring nodes before moving to the next level of nodes.
- BFS is complete and guarantees finding the shortest path if each move has the same cost.
- However its main drawback is high memory usage as it stores all nodes at the current level before moving deeper which can grow rapidly for large or complex problems.
- For Example: Which solution would BFS find to move from node S to node G if run on the graph below.





- As BFS traverses the tree shallowest node first it would always pick the shallower branch until it reaches the solution or it runs out of nodes and goes to the next branch.
- **Path**: S->D->G

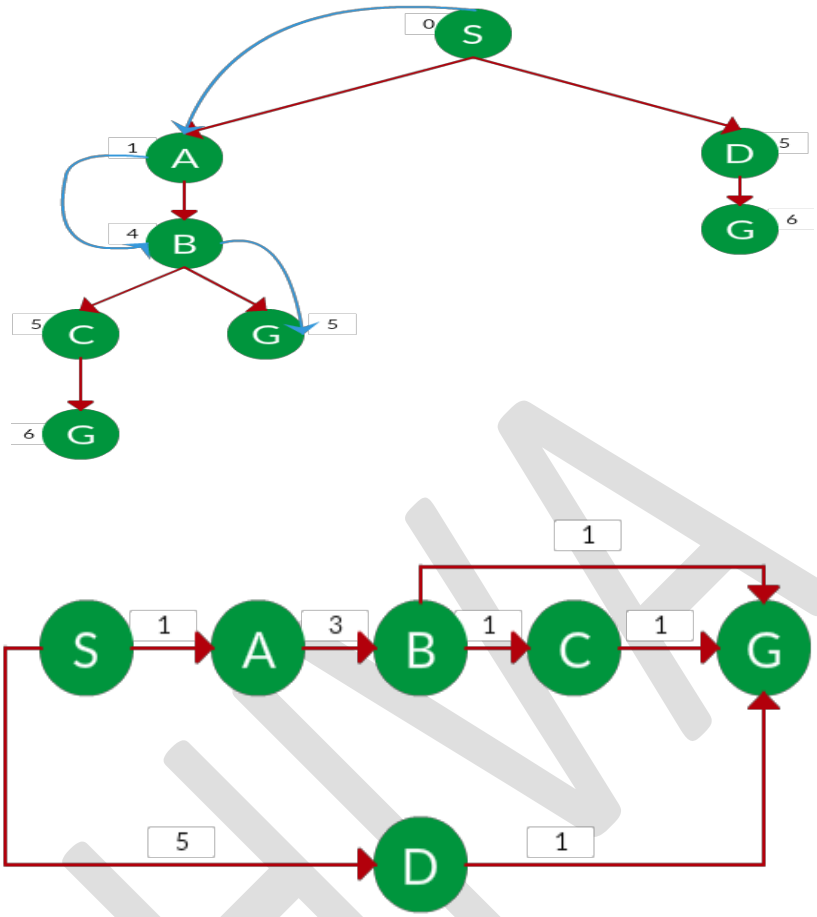
3. Uniform Cost Search(UCS):

Uniform Cost Search is similar to BFS but takes the cost of each move into account. It always expands the node with the lowest cumulative path cost from the start.

This makes UCS optimal and complete and useful when actions have different costs such as in navigation systems.

It uses a priority queue to manage the frontier and ensures the cheapest path is always chosen next.

For Example: Which solution would UCS find to move from node S to node G if run on the graph below.



- The cost of each node is the cumulative cost of reaching that node from the root and based on the UCS strategy the path with the least cumulative cost is chosen.
- Path: S->A->B->G

Informed Search Algorithms

Informed search uses domain knowledge in the form of heuristics to make smarter decisions during the search process. These heuristics estimate how close a state is to the goal guiding the search more efficiently.

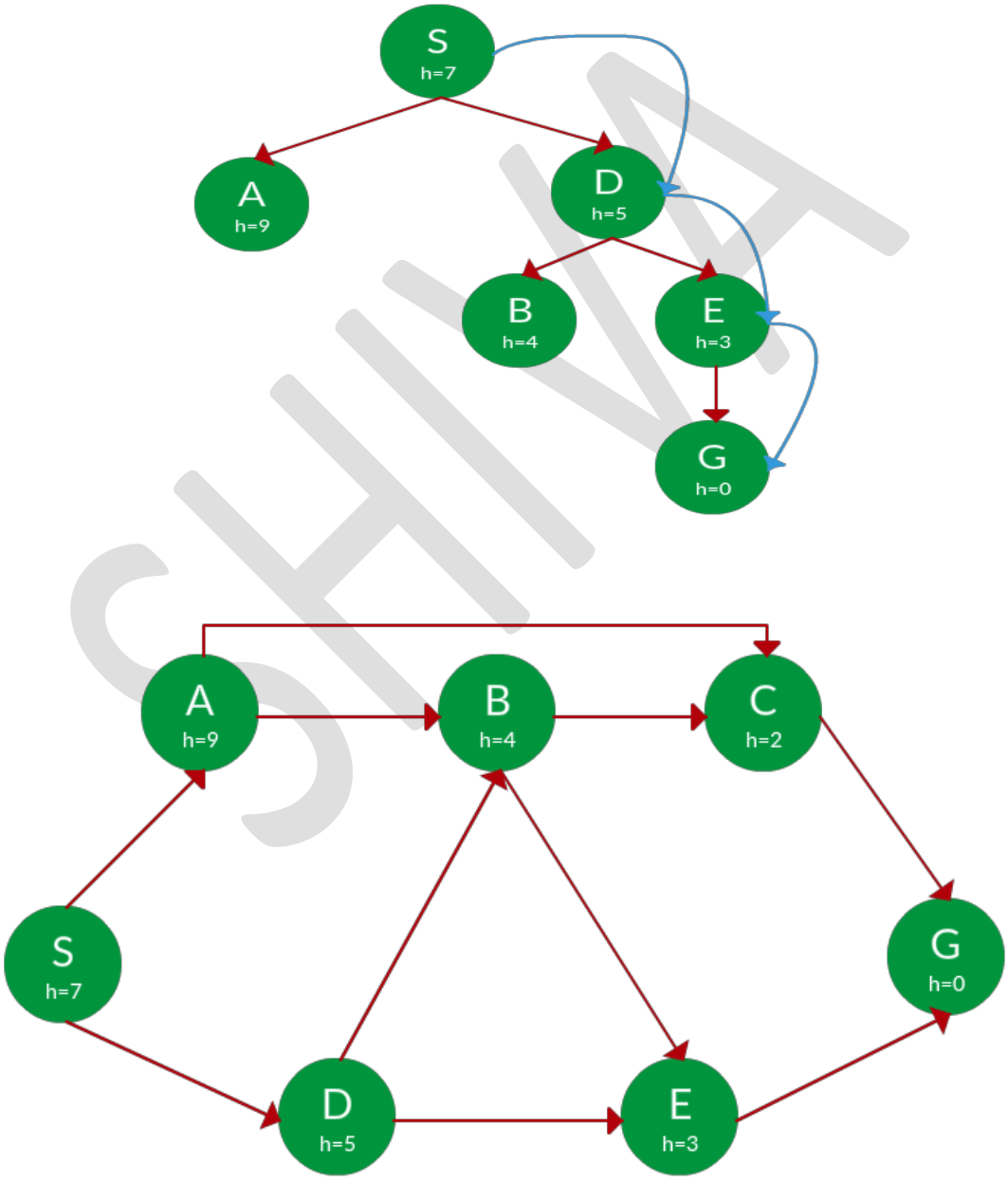
1. Greedy Search

Greedy search is a heuristic based algorithm that selects the path which appears to lead most directly toward the goal.

It makes decisions based solely on the estimated cost from the current node to the goal **(h(n))** ignoring the cost already taken to reach the current node **(g(n)).**

This approach makes greedy search fast and memory efficient as it focuses only on immediate gains.

For Example: Find the path from S to G using greedy search, the heuristic values h of each node below the name of the node.



- Starting from S we can traverse to A(h=9) or D(h=5). We choose D as it has the lower heuristic cost.
- Now from D we can move to B(h=4) or E(h=3). We choose E with a lower heuristic cost.
- Finally from E we go to G(h=0). This entire traversal is shown in the search tree below, in blue.
- Path: S->D->E->G

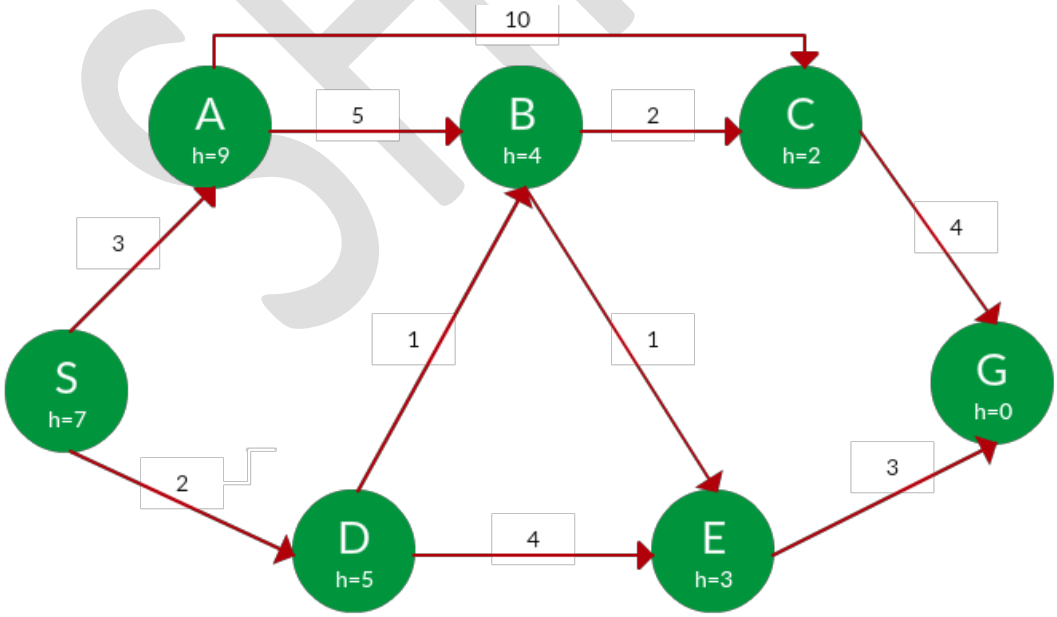
2. A* Tree Search:

A* tree search also uses the f(n) = g(n) + h(n) evaluation but treats the search space as a tree which means it doesn't track already visited nodes. Every path is explored independently even if it leads to the same state.

This can result in duplicate work and a larger search space, especially in graphs with cycles.

Although it's simpler to implement A* tree search may be less efficient and is typically used when the search structure is naturally a tree or when memory constraints prevent maintaining a closed list.

For Example: Find the path to reach from S to G using A* search.



- Starting from S the algorithm computes g(x) + h(x) for all nodes in the fringe at each step choosing the node with the lowest sum. The entire work is shown in the table below. It search like:

- Starting from S the algorithm computes g(x) + h(x) for all nodes in the fringe at each step choosing the node with the lowest sum. The entire work is shown in the table below. It search like:

Path	h(x)	g(x)	f(x)
S	7	0	7
S -> A	9	3	12
S -> D	5	2	7
S -> D -> B	4	2 + 1 = 3	7
S -> D -> E	3	2 + 4 = 6	9
S -> D -> B -> C	2	3 + 2 = 5	7
S -> D -> B -> E	3	3 + 1 = 4	7
S -> D -> B -> C -> G	0	5 + 4 = 9	9
S -> D -> B -> E -> G	0	4 + 3 = 7	7

- **Path**: S->D->B-> G-> E and Cost: 7

Comparison of Different Search Algorithms

Algorithm	Time Complexity	Space Complexity	Complete	Optimal
Breadth First Search	O(bd)$O(bd)$	O(bd)$O(bd)$	Yes	Yes (if step cost is same)
Depth First Search	O(bd)$O(bd)$	O(d)$O(d)$	No	No
Uniform Cost Search	O(b1+C∗ϵ)$O(b1+\epsilon C*)$	O(b1+C∗ϵ)$O(b1+\epsilon C*)$	Yes	Yes
Greedy Search	O(bm)$O(bm)$	O(bm)$O(bm)$	No	No
A* Tree Search	O(bd)$O(bd)$	O(bd)$O(bd)$	Yes (if h is admissible)	Yes (if h is admissible)
A* Graph Search	O(bd)$O(bd)$	O(bd)$O(bd)$	Yes (if h is admissible)	Yes (if h is admissible)

Game Playing in Artificial Intelligence

Game playing has always been a fascinating domain for artificial intelligence (AI). From the early days of computer science to the current era of advanced deep learning systems, games have served as benchmarks for AI development. They offer structured environments with clear rules, making them ideal for training algorithms to solve complex problems. With AI's ability to learn, adapt, and make strategic decisions, it is now becoming an essential player in various gaming domains, reshaping how we experience and interact with games.

In this article, we explore how AI is transforming game playing, its underlying techniques, key milestones, and future trends in the intersection of gaming and AI.

What is Game Playing in Artificial Intelligence?

Game Playing is an important domain of artificial intelligence. Games don't require much knowledge; the only knowledge we need to provide is the rules, legal moves and the conditions of winning or losing the game. Both players try to win the game. So, both of them try to make the best move possible at each turn. Searching techniques like BFS(Breadth First Search) are not accurate for this as the branching factor is very high, so searching will take a lot of time. Game playing in AI is an active area of research and has many practical applications, including game development, education, and military training. By simulating game playing scenarios, AI algorithms can be used to develop more effective decision-making systems for real-world applications.

The most common search technique in game playing is Minimax search procedure. It is depth-first depth-limited search procedure. It is used for games like chess and tic-tac-toe.

The Minimax Search Algorithm

One of the most common search techniques in game playing is the Minimax algorithm, which is a depth-first, depth-limited search procedure. Minimax is commonly used for games like chess and tic-tac-toe.

Key Functions in Minimax:

MOVEGEN: Generates all possible moves from the current position.

STATICEVALUATION: Returns a value based on the quality of a game state from the perspective of two players.

In a two-player game, one player is referred to as PLAYER1 and the other as PLAYER2. The Minimax algorithm operates by backing up values from child nodes to their parent nodes. PLAYER1 tries to maximize the value of its moves, while PLAYER2 tries to minimize the value of its moves. The algorithm recursively performs this procedure at each level of the game tree.

Example of Minimax:

Figure 1: Before backing up values

(The diagram illustrates the game tree before Minimax values are propagated upward.)

Figure 2: After backing up values

The game starts with PLAYER1. The algorithm generates four levels of the game tree. The values for nodes H, I, J, K, L, M, N, and O are provided by the STATICEVALUATION function. Level 3 is a maximizing level, so each node at this level takes the maximum value of its children. Level 2 is a minimizing level, where each node takes the minimum value of its children. After this process, the value of node A is calculated as 23, meaning that PLAYER1 should choose move C to maximize the chances of winning.

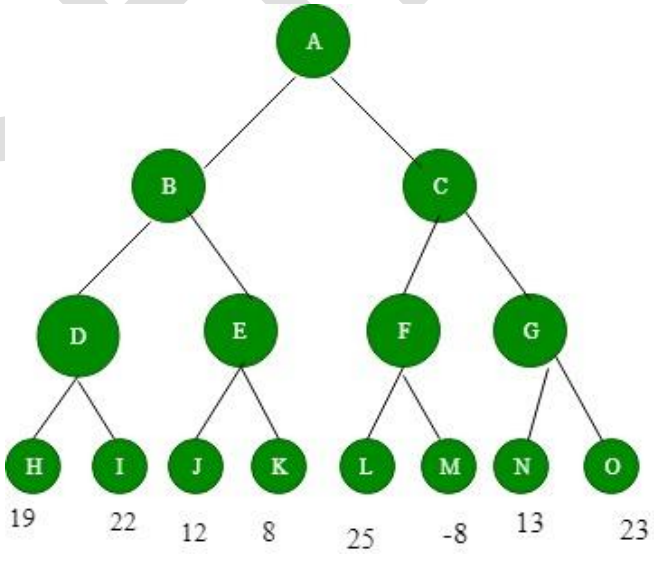
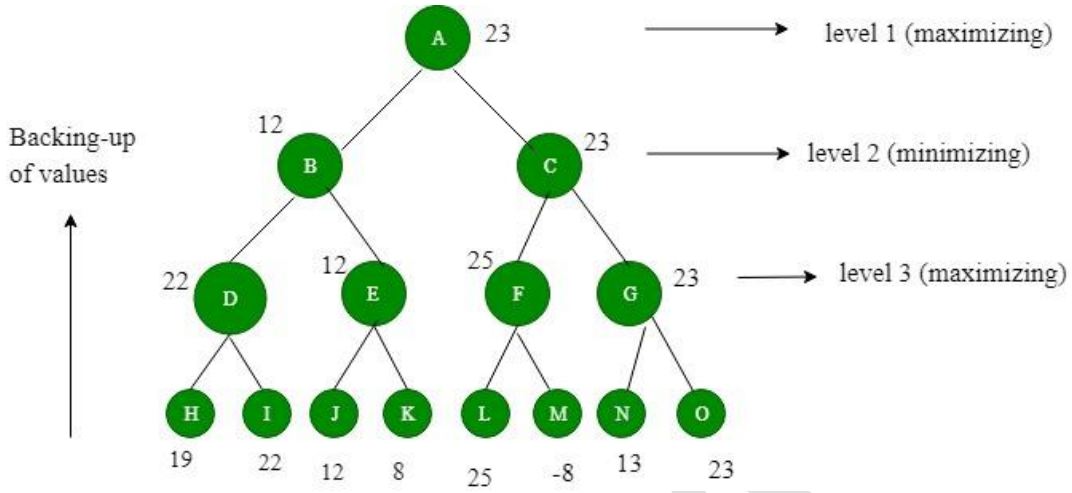


Figure 1: Before backing-up of values



Advantages of Game Playing in Artificial Intelligence

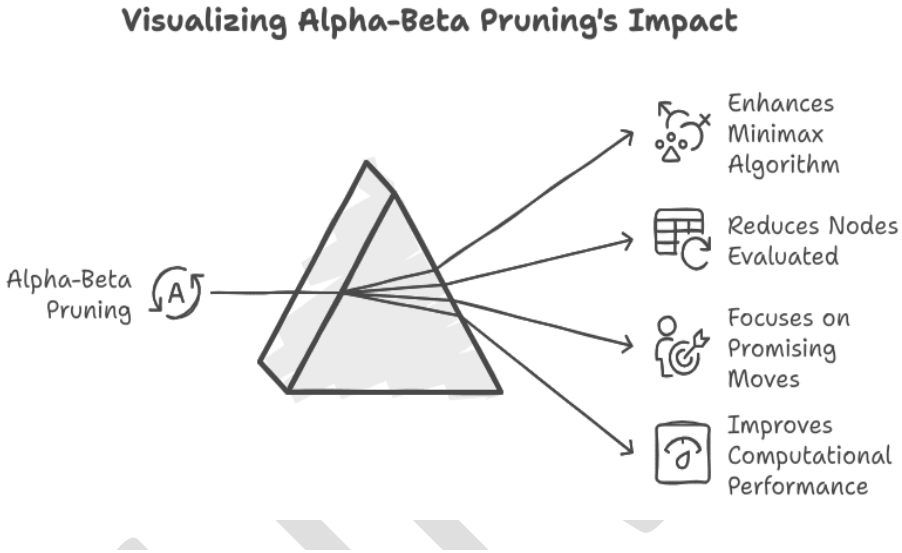
- **Advancement of AI:** Game playing has been a driving force behind the development of artificial intelligence and has led to the creation of new algorithms and techniques that can be applied to other areas of AI.
- **Education and training:** Game playing can be used to teach AI techniques and algorithms to students and professionals, as well as to provide training for military and emergency response personnel.
- **Research:** Game playing is an active area of research in AI and provides an opportunity to study and develop new techniques for decision-making and problem-solving.
- **Real-world applications:** The techniques and algorithms developed for game playing can be applied to real-world applications, such as robotics, autonomous systems, and decision support systems.

Disadvantages of Game Playing in Artificial Intelligence

- **Limited scope:** The techniques and algorithms developed for game playing may not be well-suited for other types of applications and may need to be adapted or modified for different domains.
- **Computational cost:** Game playing can be computationally expensive, especially for complex games such as chess or Go, and may require powerful computers to achieve real-time performance.

Alpha Beta Pruning in AI

- Alpha Beta Pruning optimizes AI decision-making by reducing unnecessary computations in the minimax algorithm. Learn its workings, benefits, and real-world applications in gaming and beyond.



How Alpha Beta Pruning Works

The key idea behind Alpha Beta Pruning is to avoid evaluating branches of the game tree that cannot influence the final decision based on the values already discovered during the search. It achieves this using two values: **Alpha** and **Beta**.

Alpha and Beta Values

- **Alpha** represents the best (highest-value) value that the maximizing player (usually the AI) can guarantee so far. It acts as a lower bound. The initial value of alpha is $-\infty$.
- **Beta** represents the best (lowest-value) value that the minimizing player (the opponent) can guarantee so far. It acts as an upper bound. The initial value of alpha is $+\infty$.

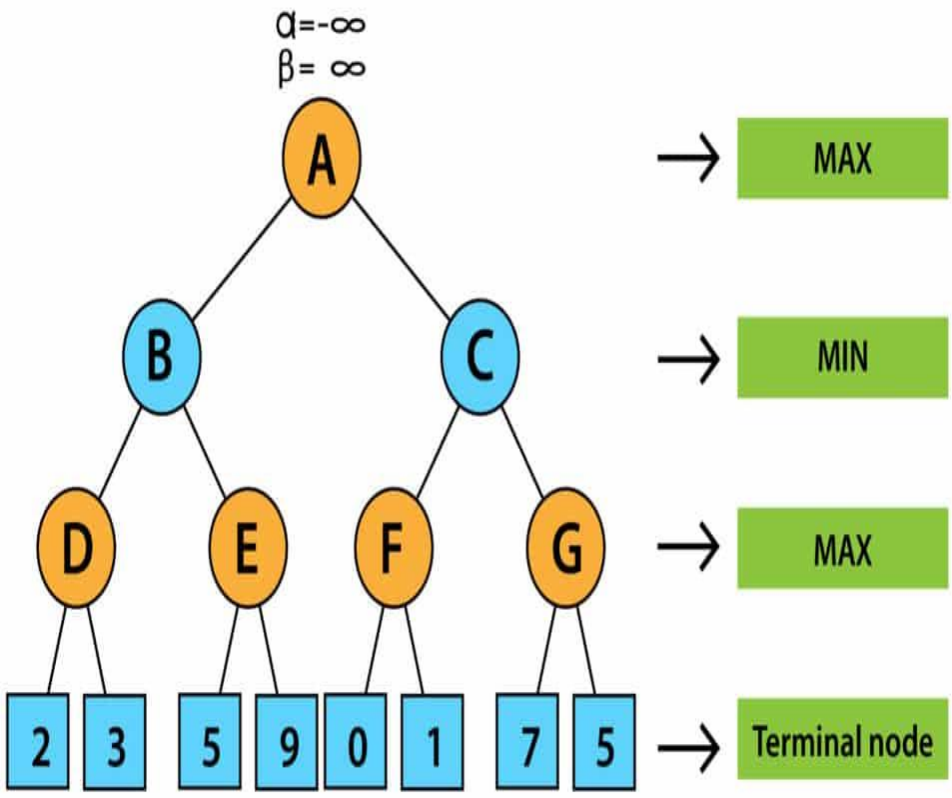
The Pruning Process

- As the AI explores the tree, it keeps track of Alpha and Beta values. When exploring a node, it compares the node's value against these values.

- If, at any point, **Alpha** becomes greater than or equal to **Beta**; it means the current branch will not affect the final decision because the opponent will avoid this path in favor of a better one. As a result, this branch is pruned, and the algorithm moves on to the next branch.
- This process allows the algorithm to skip large parts of the tree, significantly reducing the number of nodes to be evaluated.

Working of Alpha-beta Pruning

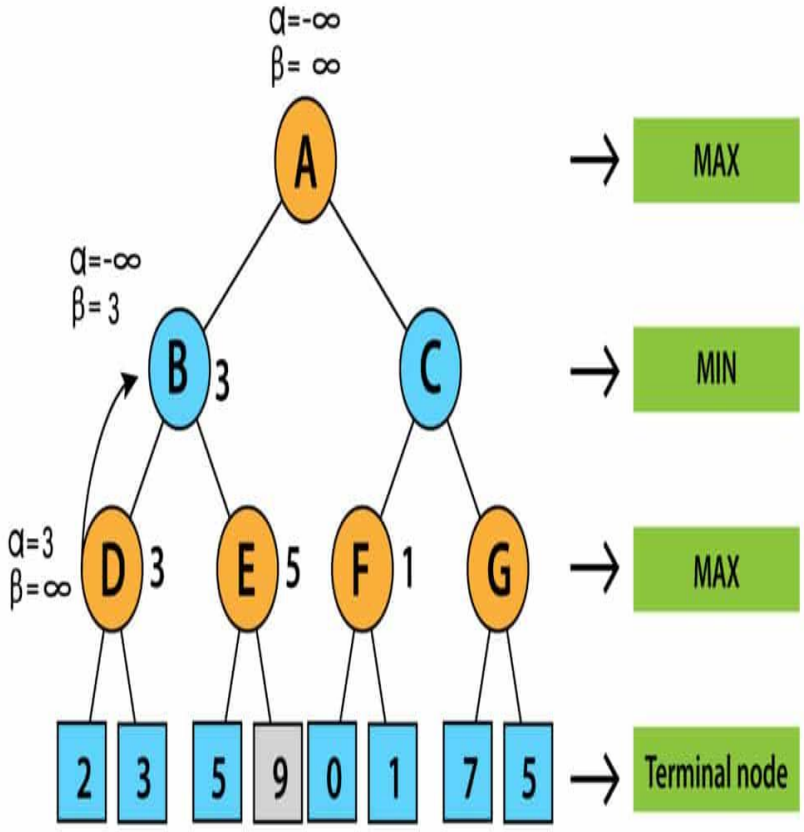
1. First, we will take care of the first move. So initially, we will define the worst case $\alpha = -\infty$ and $\beta = +\infty$. If alpha is greater than or equal to beta, we will prune the node.



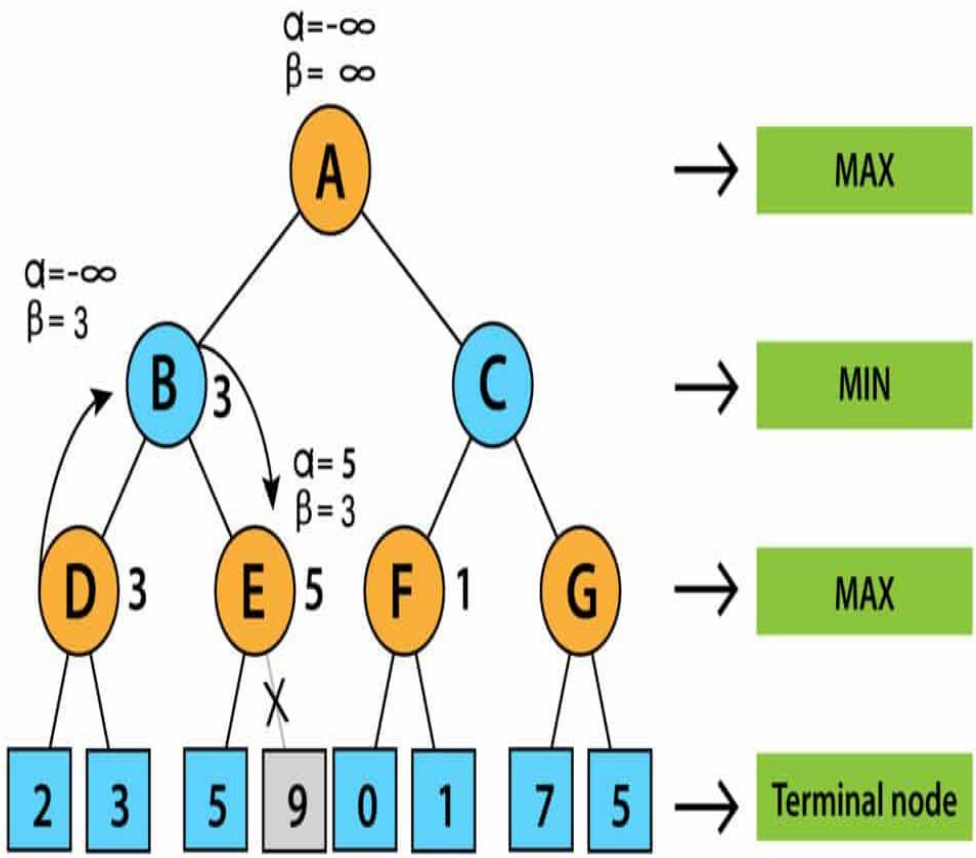
2. We didn't prune it since the initial value of alpha is less than beta. Now it's turn for MAX. Therefore, we will calculate the value of alpha at node D. At node D, the value of alpha will be max (2, 3) = 3.

3. Now, node B's turn is MIN. That means that the value of alpha beta at node B will be min (3, ∞). Therefore, alpha will be – ∞ and beta will be 3 at node B.

Next, algorithms will pass the values of α= -∞ and β= 3 to the next successor of Node B, that is node E.



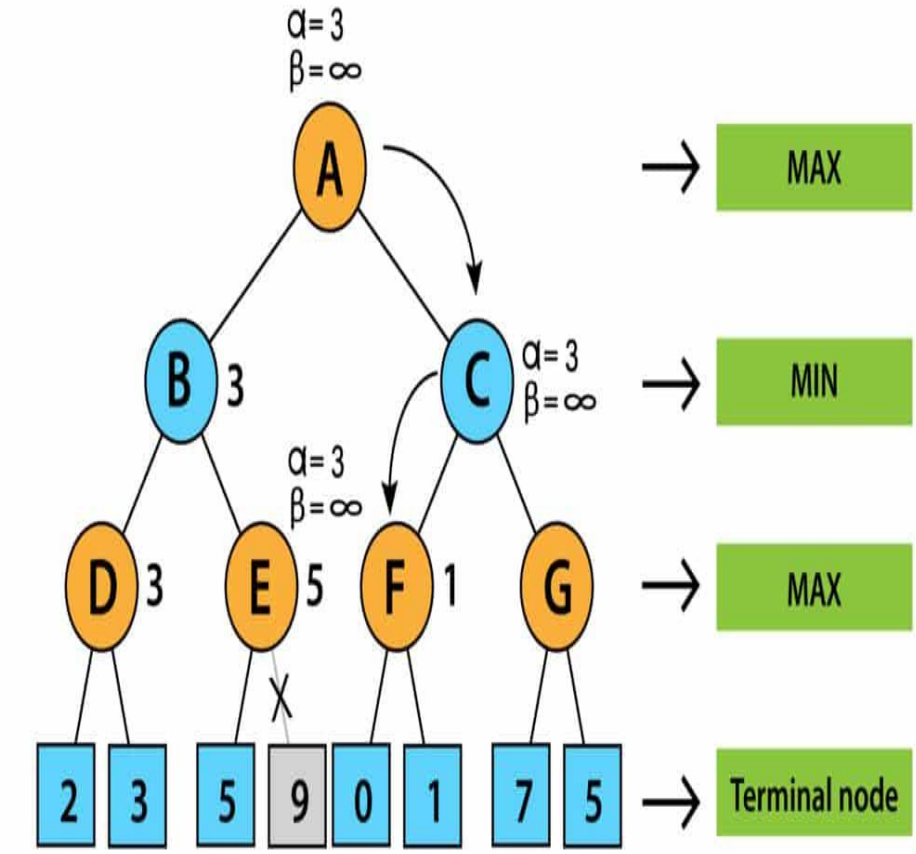
4. Now it's turn for MAX. Therefore, we will search for MAX at node E. The value of alpha at E is – ∞ and will be compared with 5. So, MAX (- ∞, 5) will be 5. Thus, at node E, alpha = 5, Beta = 5.



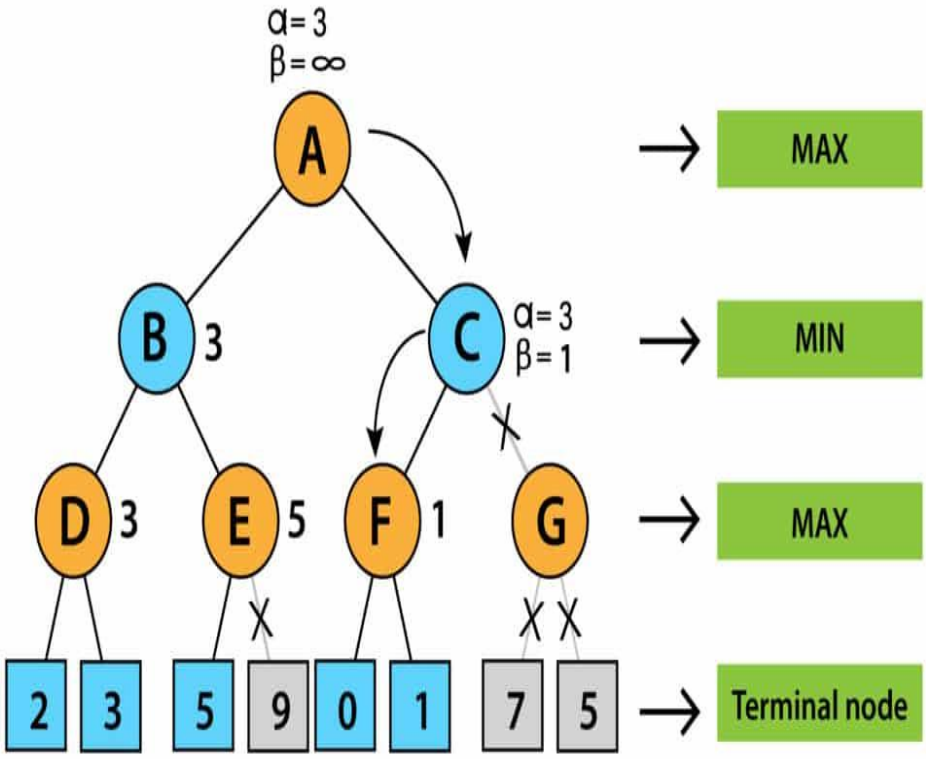
We now know that alpha is greater than beta and is also a pruning condition, so we can prune the right successor of node E, and the algorithm will not be visited, and the value of node E will be 5.

5. In the next step, the algorithm again comes from node B to node A. The alpha of node A is changed to the max value of MAX(- ∞, 3). Now, alpha and beta at node A are (3, + ∞) and will be transmitted to node C. The same values will be transferred to node F.

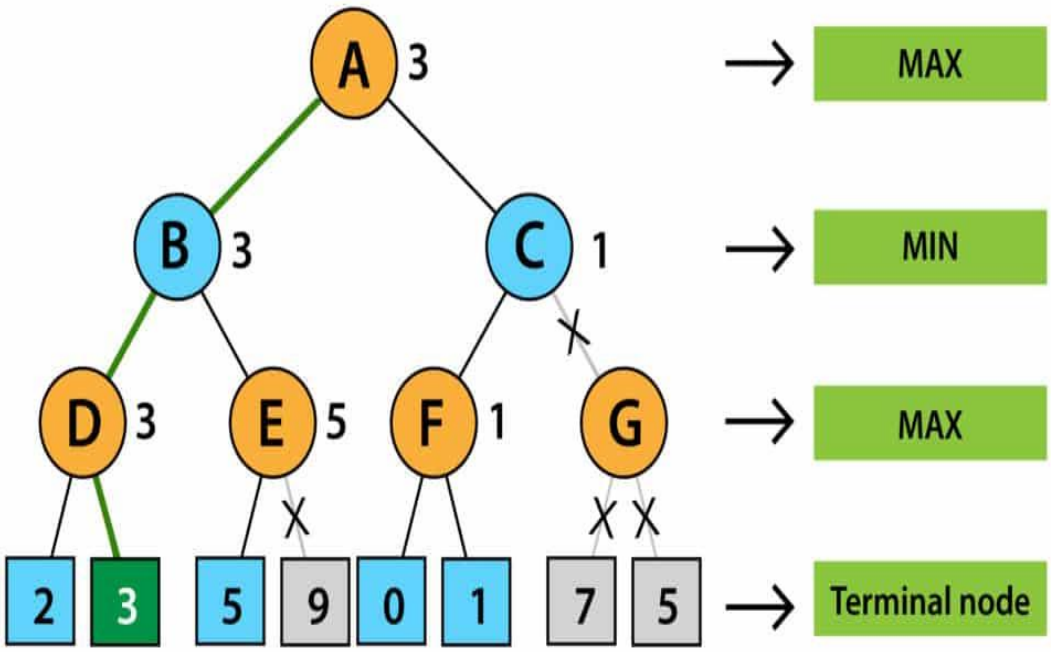
6. The value of alpha will be compared to the left branch, which is 0 at node F. Now, MAX (0, 3) will be 3 and is compared with the right child (1) and MAX(3, 1) = 3 still α is 3, but the node value of F will be 1.



7. Afterward, node F will pass the node value 1 to C and compare it to the beta value at C. Now it's turn for MIN. So, MIN (+ ∞, 1) will be 1. Now, at node C, α = 3, β= 1, and alpha is greater than beta, which again satisfies the pruning condition. Then, the successor of node C, G, will be pruned, and the algorithm did not calculate the whole subtree G.



Now, C will return its node value to A, and A will calculate MAX(1, 3), resulting in 3.



This above-represented tree is the final tree that has nodes that are computed and nodes that are not computed. Therefore, in this example, the optimal value of the maximizer would be 3.

Key Benefits of Alpha Beta Pruning in AI

When the game trees are complex, some important advantages of Alpha Beta Pruning for AI decision-making algorithms come into play.

1. Improved Efficiency

By pruning unnecessary branches, Alpha Beta Pruning significantly reduces the amount of computation required. The algorithm can focus on evaluating only the most relevant parts of the tree, making decision-making much faster.

2. Reduced Time Complexity

Without pruning, the minimax algorithm has a time complexity of O(b^d), where b represents the branching factor (the number of possible moves) and d is

the depth of the tree. With Alpha Beta Pruning, the time complexity can be reduced to O(b^(d/2)) in the best case, enabling the AI to explore deeper levels of the tree much more efficiently.

3. Scalability

As AI applications become more complex and require deeper search trees (e.g., in chess or Go), **Alpha Beta Pruning** makes exploring these larger search spaces feasible. The ability to prune unnecessary branches means that AI systems can handle more complex decision-making tasks in real time.

Applications of Alpha Beta Pruning in AI

Alpha Beta Pruning is widely used in AI applications where decision trees need to be explored efficiently. Some key applications include:

1. Game AI

In strategic games like chess, checkers, or Go, **Alpha Beta Pruning** becomes crucial for enabling AI to evaluate millions of possible moves in real-time.

Games like **Stockfish** (chess) and **AlphaGo** (Go) use **Alpha Beta Pruning** as a core component of their decision-making algorithms, enabling them to compete with human players at the highest level.

2. Autonomous Systems and Robotics

Decision trees are used frequently in robotics automation for real time decisions on movement, navigation, and task execution. Alpha Beta Pruning aids in quickly evaluating different paths and strategies through robots, which helps them respond faster in a dynamic environment.

3. Models on Financial and Optimization

Decision trees are also used in financial forecasting, portfolio optimization, and supply chain management through AI systems, especially in evaluating different scenarios and making predictions. To make more timely decisions, these systems could use Alpha Beta Pruning to process large datasets.

Constraint Satisfaction Problems (CSP) in Artificial Intelligence:

A Constraint Satisfaction Problem is a mathematical problem where the solution must meet a number of constraints. In CSP the objective is to assign values to variables such that all the constraints are satisfied. Many AI applications use CSPs to solve decision-making problems that involve managing or arranging resources under strict guidelines. Common applications of CSPs include:

Scheduling: It assigns resources like employees or equipment while respecting time and availability constraints.

Planning: Organize tasks with specific deadlines or sequences.

Resource Allocation: Distributing resources efficiently without overuse

Components of Constraint Satisfaction Problems CSPs are composed of three key elements:

1. Variables: These are the things we need to find values for. Each variable represents something that needs to be decided. For example, in a Sudoku puzzle each empty cell is a variable that needs a number. Variables can be of different types like yes/no choices (Boolean), whole numbers (integers) or categories like colors or names.

2. Domains: This is the set of possible values that a variable can have. The domain tells us what values we can choose for each variable. In Sudoku the domain for each cell is the numbers 1 to 9 because each cell must contain one of these numbers. Some domains are small and limited while others can be very large or even infinite.

3. Constraints: These are the rules that restrict how variables can be assigned values. Constraints define which combinations of values are allowed. There are different types of constraints:

Unary constraints apply to a single variable like "this cell cannot be 5".

Binary constraints involve two variables like "these two cells cannot have the same number".

Higher-order constraints involve three or more variables like "each row in Sudoku must have all numbers from 1 to 9 without repetition".

Types of Constraint Satisfaction Problems

CSPs can be classified into different types based on their constraints and problem characteristics:

Binary CSPs: In these problems each constraint involves only two variables. Like in a scheduling problem the constraint could specify that task A must be completed before task B.

Non-Binary CSPs: These problems have constraints that involve more than two variables. For instance in a seating arrangement problem a constraint could state that three people cannot sit next to each other.

Hard and Soft Constraints: Hard constraints must be strictly satisfied while soft constraints can be violated but at a certain cost. This is often used in real-world applications where not all constraints are equally important.

Representation of Constraint Satisfaction Problems (CSP)

In CSP it involves the interaction of variables, domains and constraints. Below is a structured representation of how CSP is formulated:

Finite Set of Variables

1. **Finite Set of Variables** (V1,V2,...,Vn)(*V*1,*V*2,...,*Vn*)**:** The problem consists of a set of variables each of which needs to be assigned a value that satisfies the given constraints.
2. **Non-Empty Domain for Each Variable** (D1,D2,...,Dn)(*D*1,*D*2,...,*Dn*)**:** Each variable has a domain a set of possible values that it can take. For example, in a Sudoku puzzle the domain could be the numbers 1 to 9 for each cell.
3. **Finite Set of Constraints** (C1,C2,...,Cm)(*C*1,*C*2,...,*Cm*)**:** Constraints restrict the possible values that variables can take. Each constraint defines a rule or relationship between variables.
4. **Constraint Representation:** Each constraint Ci*Ci* is represented as a pair of (scope, relation) where:

- **Scope:** The set of variables involved in the constraint.
- **Relation:** A list of valid combinations of variable values that satisfy the constraint.

Example: Let's say you have two variables V1*V*1 and V2*V*2. A possible constraint could be V1≠V2*V*1□=*V*2, which means the values assigned to these variables must not be equal. There:

- **Scope:** The variables V1*V*1 and V2*V*2.
- **Relation:** A list of valid value combinations where V1*V*1 is not equal to V2*V*2.

Some relations might include explicit combinations while others may rely on abstract relations that are tested for validity dynamically.

Solving Constraint Satisfaction Problems Efficiently

CSP use various algorithms to explore and optimize the search space ensuring that solutions meet the specified constraints. Here's a breakdown of the most commonly used CSP algorithms:

1. Backtracking Algorithm

The backtracking algorithm is a depth-first search method used to systematically explore possible solutions in CSPs. It operates by assigning values to variables and backtracks if any assignment violates a constraint.

How it works:

The algorithm selects a variable and assigns it a value.

It recursively assigns values to subsequent variables.

If a conflict arises i.e a variable cannot be assigned a valid value then algorithm backtracks to the previous variable and tries a different value.

The process continues until either a valid solution is found or all possibilities have been exhausted**.**

This method is widely used due to its simplicity but can be inefficient for large problems with many variables.

2. Forward-Checking Algorithm

The forward-checking algorithm is an enhancement of the backtracking algorithm that aims to reduce the search space by applying local consistency checks.

How it works:

For each unassigned variable the algorithm keeps track of remaining valid values.

Once a variable is assigned a value local constraints are applied to neighboring variables and eliminate inconsistent values from their domains.

If a neighbor has no valid values left after forward-checking the algorithm backtracks.

This method is more efficient than pure backtracking because it prevents some conflicts before they happen reducing unnecessary computations.

3. Constraint Propagation Algorithms

Constraint propagation algorithms further reduce the search space by enforcing local consistency across all variables.

How it works:

Constraints are propagated between related variables.

Inconsistent values are eliminated from variable domains by using information gained from other variables.

These algorithms filter the search space by making inferences and by remove values that would led to conflicts.

Constraint propagation is used along with other CSP methods like backtracking to make the search faster.

Solving Sudoku with Constraint Satisfaction Problem (CSP) Algorithms

Step 1: Define the Problem (Sudoku Puzzle Setup)

The first step is to define the Sudoku puzzle as a 9x9 grid where 0 represents an empty cell. We also define a function print_sudoku to display the puzzle in a human readable format.

```
Initial Sudoku Puzzle:

5 3 0  | 0 7 0  | 0 0 0
6 0 0  | 1 9 5  | 0 0 0
0 9 8  | 0 0 0  | 0 6 0
- - - - - - - - - - - -
8 0 0  | 0 6 0  | 0 0 3
4 0 0  | 8 0 3  | 0 0 1
7 0 0  | 0 2 0  | 0 0 6
- - - - - - - - - - - -
0 6 0  | 0 0 0  | 2 8 0
0 0 0  | 4 1 9  | 0 0 5
0 0 0  | 0 8 0  | 0 7 9
```

SHIVA